EE224 Digital Systems Project

Bubble-Sort Implementation on i281 CPU

Submitted By:

Pratyush Kumar Masanta (24B1294)
Jayent Dev (24B1232)
Sailesh Kumar Sahoo (24B1208)

Instructor: Sachin B. Patkar

Indian Institute of Technology Bombay
2025-26

Contents

1. Work Division

- **Jayent Dev**
  - Codes for ALU, Flag Register, Register File
  - Final integration of all components and debuging
- **Pratyush Kumar Masanta**
  - Codes for Data Memory, Code Memory, Program Counter
  - Final integration of all components and debuging
- **Sailesh Kumar Sahoo**
  - Codes for Control Unit, Testbench, PC Update logic
  - Final integration of all components and debuging

2. i281 CPU

2.1 Introduction

The i281 CPU is a simple yet powerful 16-bit computer system designed to demonstrate efficient hardware-based computation. The architecture of the processor, including its control unit, memory system, associated modules, and all, is implemented entirely in Verilog, providing a clean and structured hardware description of the system.

4-Register System which makes the computing system provide flexibility in storing and manipulating data.

128-bit Data Memory helps in storing up to 16 8-bit register values

2.2 Structure

Overview of all the components of the CPU:

- **Switches**: These 16 bits are configured externally by user, which go to the *Code Memory*. Optionally, the lower 8 bits can go into the *Data Memory*.
- **Register File**: It contains 4 registers which can be read (by the ALU) or written.
- **Arithmetic Logic Unit(ALU)**: Depending on the *ALU Select* bits, it can perform left shifting, right shifting, addition, or subtraction/comparison. Technically it is always enabled, just that the result is ignored.
- **Flags Register**: When *Write Enable* is asserted, it updates 4 flags as directed from ALU, namely *Carry Flag*, *Overflow Flag*, *Negative Flag*, and *Zero Flag*. Only 7 OPCODEs update the Flags Register.
- **Program Counter**: 6-bit parallel access register which displays the current instruction address of *Code Memory*. When *Write Enable* is asserted, it updates with the address received from *PC Mux*.

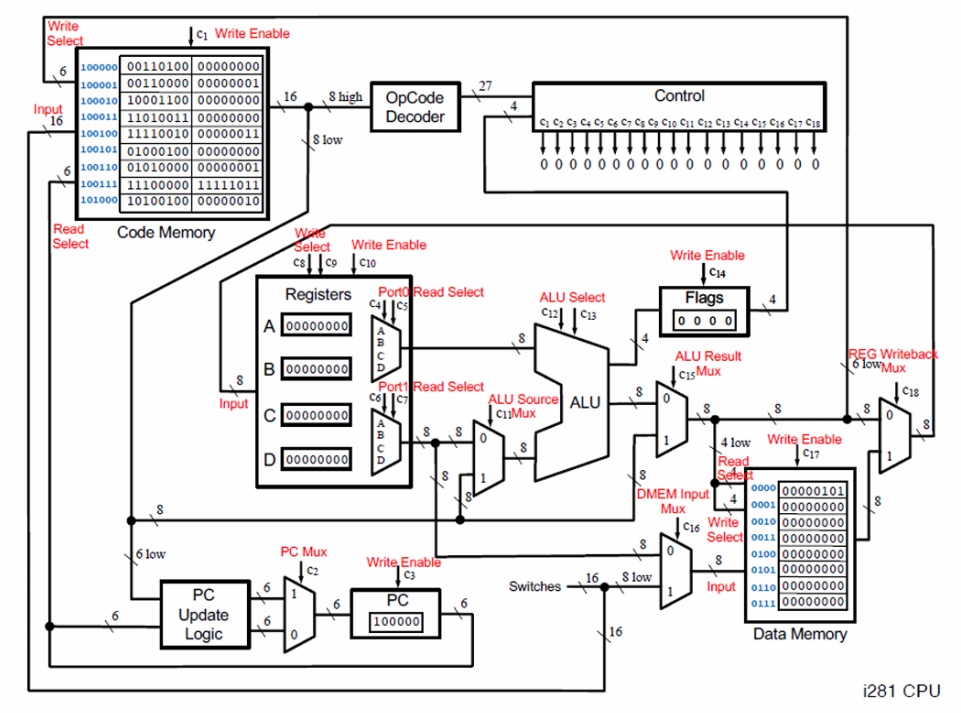


Figure 1: i281 I/O architecture

- **Program Counter Update Logic**: It gives 2 outputs: +1 incremented instruction address, and +1 incremented address offset by the lower 6 bits of current Code Memory data.
- **Data Memory (DMEM)**: Register File with 16 registers, each 8 bits wide. Total *16 bytes* of readable/writable memory.
- **Code Memory (CMEM/IMEM)**: Register File with 64 registers, with 32 registers *BIOS* (Read Only) memory and rest 32 registers *USER* (Read Only in User Mode, Read/Write in BIOS Mode). Total *128 bytes* of memory.
- **Control Unit**: Generates all the 18 *control signals* based on the current OPCODE and break flags.
- **OPCODE Decoder**: Based on the higher 8 bits of the current code memory, outputs a *23 bit One-Hot encoded* OPCODE, along with 4 other bits (not one-hot encoded).

3. OPCODEs

OPCODE	**Purpose**
NOOP	NO OPeration
INPUTC	INPUT into Code memory
INPUTCF	INPUT into Code memory with oFfset
INPUTD	INPUT into Data memory
INPUTDF	INPUT into Data memory with oFfset
MOVE	MOVE the contents of one register into another
LOADI	LOAD Immediate value
LOADP	LOAD Pointer address
ADD	ADD two registers
ADDI	ADD an Immediate value to a register
SUB	SUBtract two registers
SUBI	SUBtract an Immediate value from a register
LOAD	LOAD from a data memory address into a register
LOADF	LOAD with an oFfset specified by another register
STORE	STORE a register into a data memory address
STOREF	STORE with an oFfset specified by another register
SHIFTL	SHIFT Left all bits in a register
SHIFTR	SHIFT Right all bits in a register
CMP	CoMPare the values in two registers
JUMP	JUMP unconditionally to a specified address
BRE	BRanch if Equal
BRZ	BRanch if Zero
BRNE	BRanch if Not Equal
BRNZ	BRanch if Not Zero
BRG	BRanch if Greater
BRGE	BRanch if Greater than or Equal

4. Detailed Implementation of Different Sections of the CPU

4.1 Program Counter

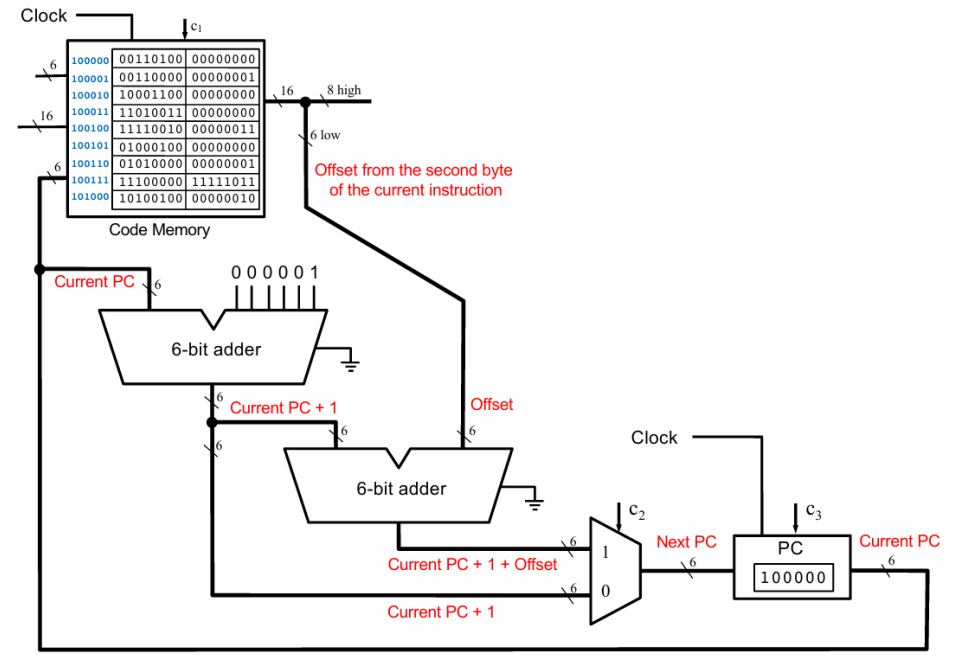


Figure 2: PC Update Logic

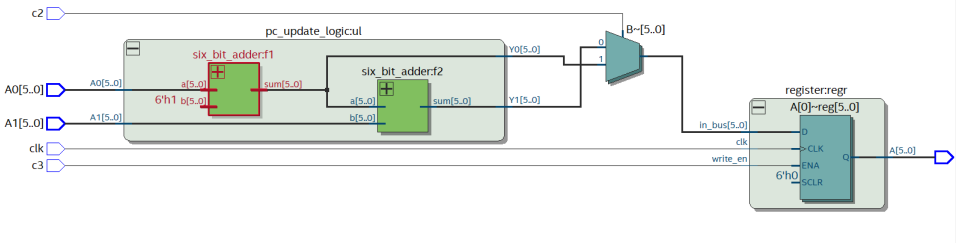


Figure 3: Netlist View of Program Counter

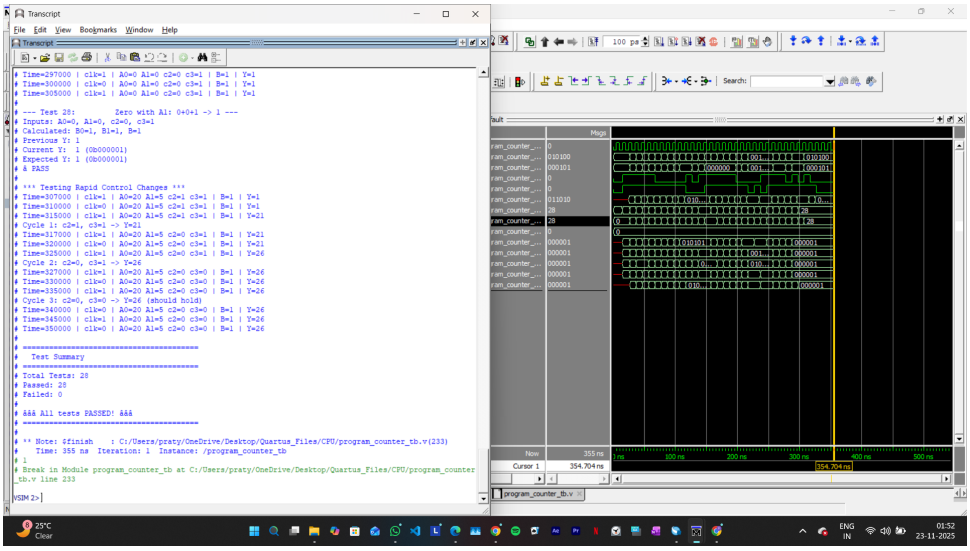


Figure 4: Simulation on ModelSim using an appropriate testbench

4.2 Arithmetic Logic Unit

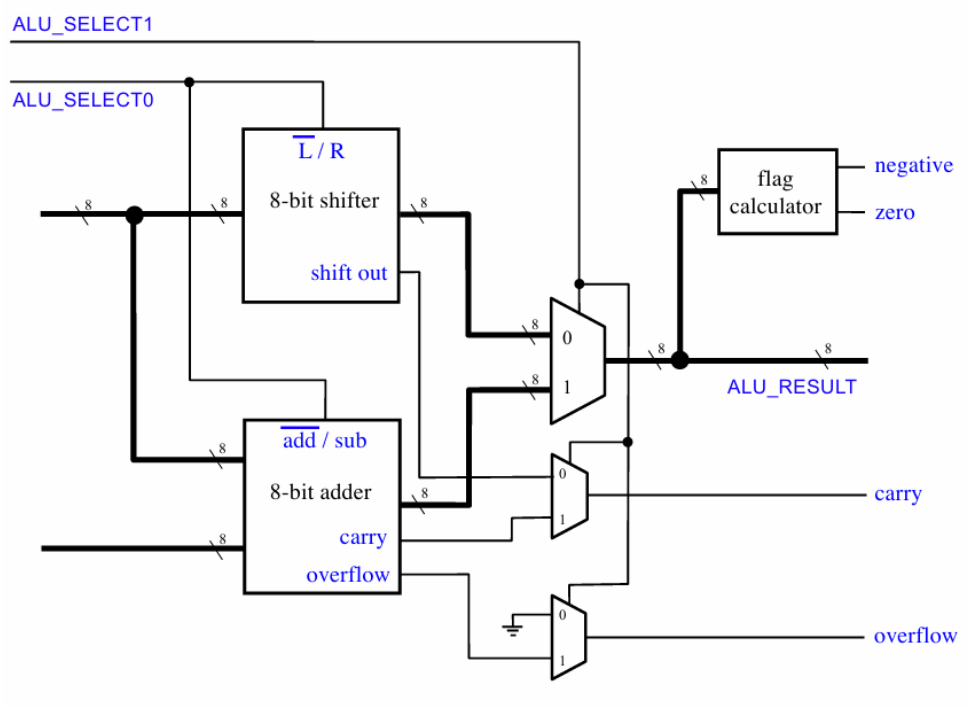


Figure 5: Arithmetic Logic Unit

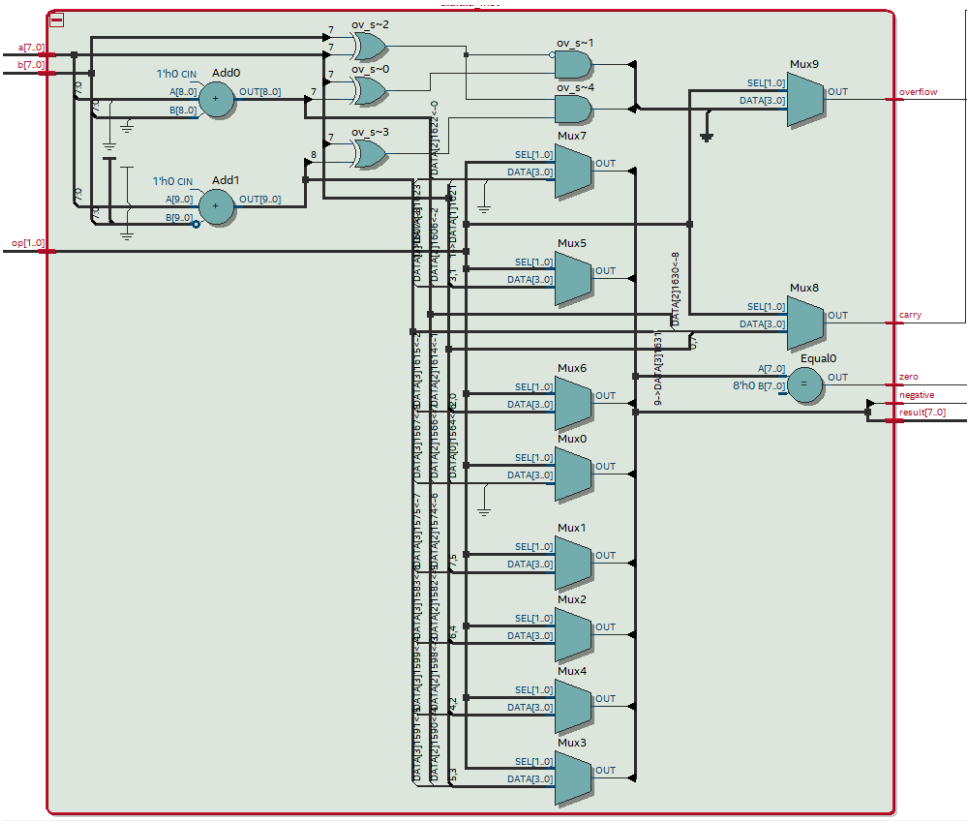


Figure 6: Netlist View of ALU

4.3 OPCODE Decoder

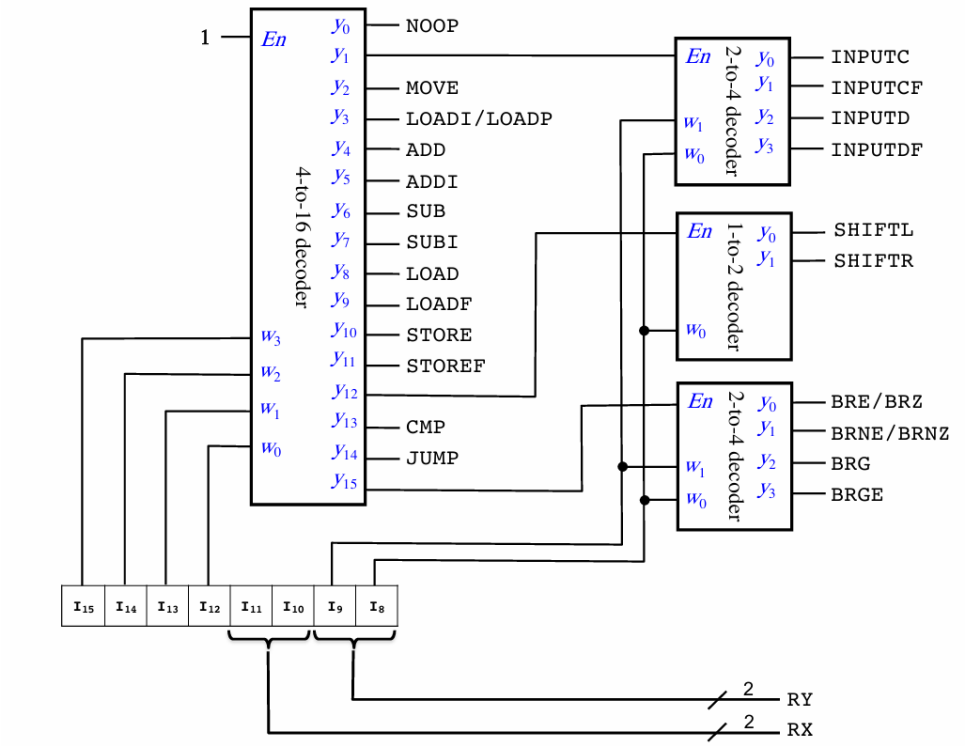


Figure 7: Opcode Decoder

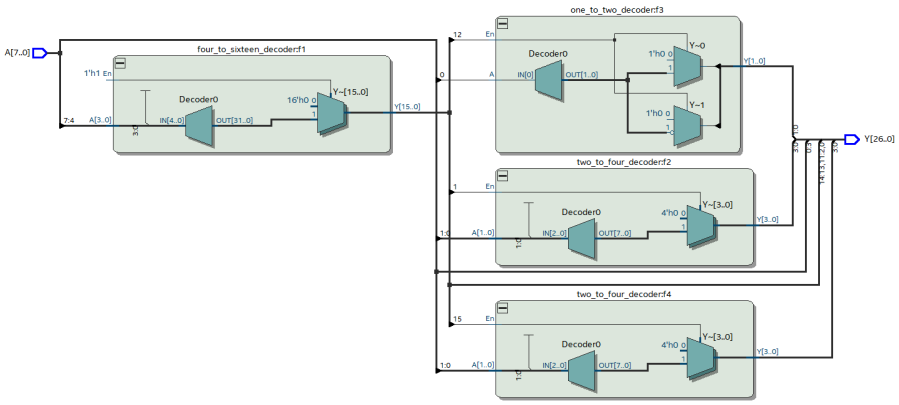


Figure 8: Netlist View of Opcode Decoder

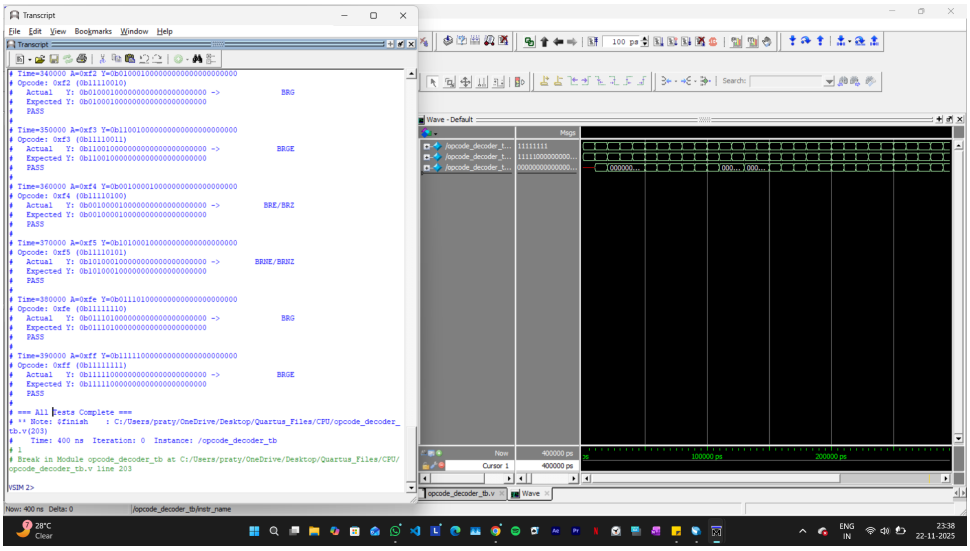
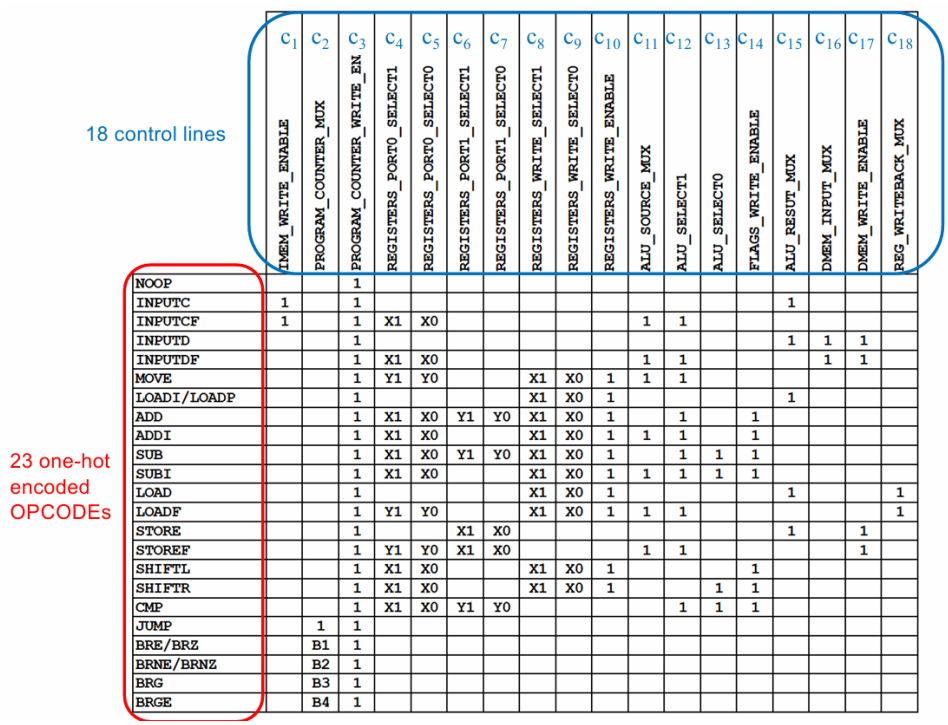


Figure 9: Simulation on ModelSim using an appropriate testbench

4.4 Control Unit



	c1 IMEM_WRITE_ENABLE	c2 PROGRAM_COUNTER_MUX	c3 PROGRAM_COUNTER_WRITE_EN	c4 REGISTERS_PORT0_SELECT1	c5 REGISTERS_PORT0_SELECT0	c6 REGISTERS_PORT1_SELECT1	c7 REGISTERS_PORT1_SELECT0	c8 REGISTERS_WRITE_SELECT1	c9 REGISTERS_WRITE_SELECT0	c10 REGISTERS_WRITE_ENABLE	c11 ALU_SOURCE_MUX	c12 ALU_SELECT1	c13 ALU_SELECT0	c14 FLAGS_WRITE_ENABLE	c15 ALU_RESUT_MUX	c16 DMEM_INPUT_MUX	c17 DMEM_WRITE_ENABLE	c18 REG_WRITEBACK_MUX
NOOP			1															
INPUTC	1		1												1			
INPUTCF	1		1	X1	X0						1	1						
INPUTD			1												1	1	1	
INPUTDF			1	X1	X0						1	1				1	1	
MOVE			1	Y1	Y0			X1	X0	1	1	1						
LOADI/LOADP			1					X1	X0	1					1			
ADD			1	X1	X0	Y1	Y0	X1	X0	1		1		1				
ADDI			1	X1	X0			X1	X0	1	1	1		1				
SUB			1	X1	X0	Y1	Y0	X1	X0	1		1	1	1				
SUBI			1	X1	X0			X1	X0	1	1	1	1	1				
LOAD			1					X1	X0	1					1			1
LOADF			1	Y1	Y0			X1	X0	1	1	1						1
STORE			1			X1	X0								1		1	
STOREF			1	Y1	Y0	X1	X0				1	1					1	
SHIFTL			1	X1	X0			X1	X0	1				1				
SHIFTR			1	X1	X0			X1	X0	1			1	1				
CMP			1	X1	X0	Y1	Y0					1	1	1				
JUMP		1	1															
BRE/BRZ		B1	1															
BRNE/BRNZ		B2	1															
BRG		B3	1															
BRGE		B4	1															

Figure 10: Control Unit

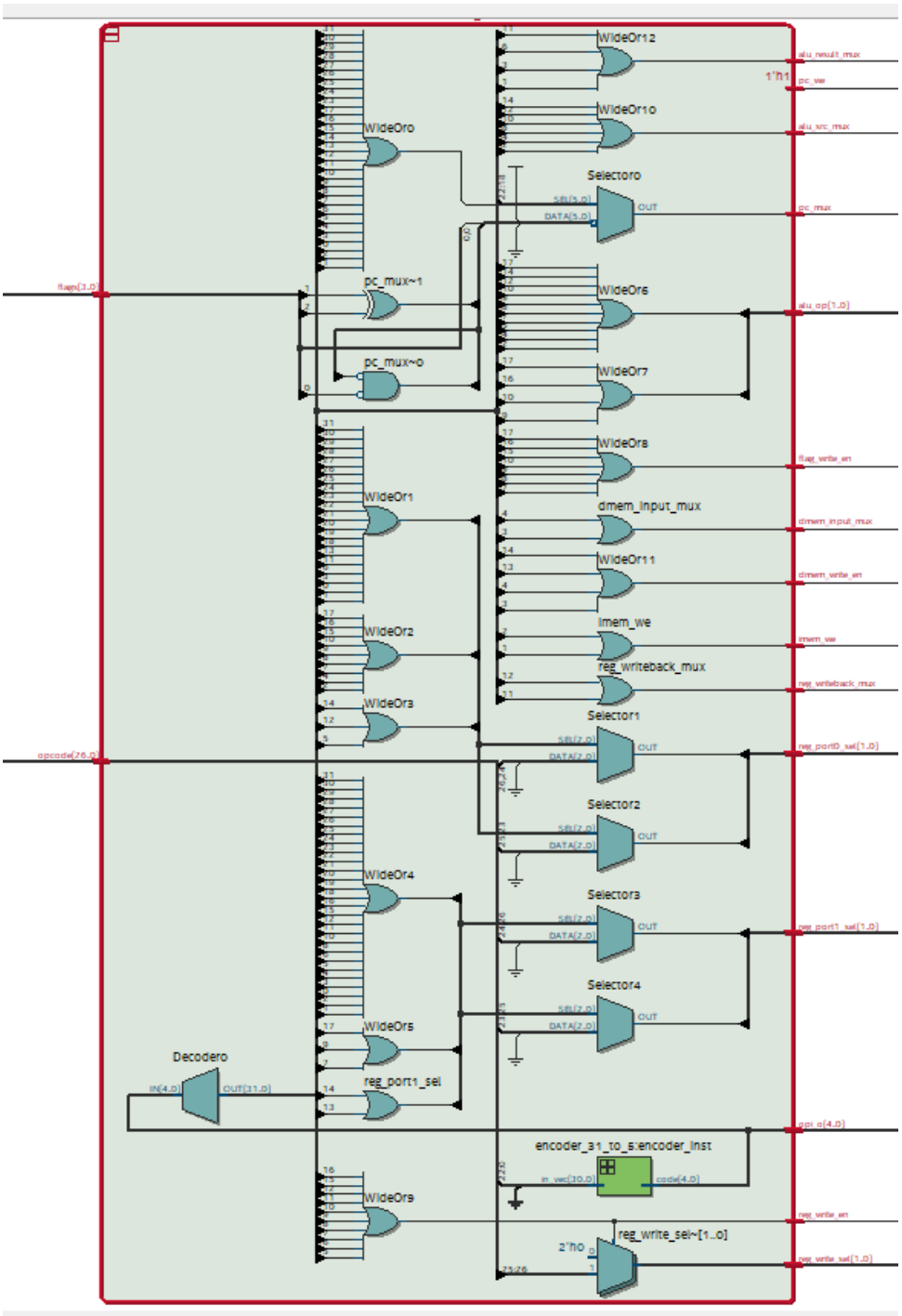


Figure 11: Netlist view of Control Unit

5. File Structure Description

- cpu.v: Top Level verilog file which instantiates all the helper modules to finally integrate the i281 CPU implementing the bubble sort algorithm
- tb_load_and_run.v: Testbench which instantiates the top level file to show the output simulation of the data memory, implementing the sorting algorithm
- register_file.v: Register File
- program_counter.v: Program Counter along with its update logic
- opcode_decoder.v: OPCODE decoder
- flag_register.v: Flags register
- enc_31_to_5.v: 31 to 5 encoder
- dec_4_to_16.v: 4 to 16 decoder
- dec_2_to_4.v: 2 to 4 decoder
- dec_1_to_2.v: 1 to 2 decoder
- data_mem.v: Data Memory
- control_unit.v: Control Unit
- code_mem.v: Code Memory
- alu.v: Arithmetic Logic Unit

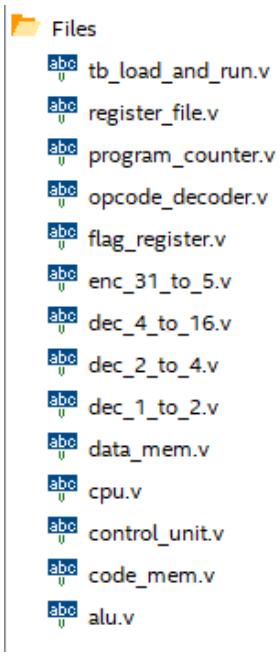


Figure 12: File Heirarchy

6. Bubble Sort Algorithm Implementation

Bubble Sort Algorithm is a simple algorithm to sort a list of numbers. We start at the beginning of the list and compare the first 2 elements. If first element is greater than the second, then we swap them, otherwise leave them as they are. Then we move to the second and third element and repeat the previous steps. This process is continued till we reach the end of the list. After one full pass, the largest element is at the end of the list. Now this entire passing algorithm is repeated again for the list except for the last element(since it is sorted already). The algorithm is completed when no swaps are needed in a complete pass.

The C language and assembly language implementation is shown in the figure:

C Version

```
int array[] = {7, 3, 2, 1, 6, 4, 5, 8};
int last = 7; // last valid index in the array
int temp;
int i, j;

int main()
{
   for (i = 0; i < last; i++)
        for (j = 0; j < last-i; j++)
               if (array[j] > array[j+1]){
                      temp = array[j];
                      array[j] = array[j+1];
                      array[j+1] = temp;
               }

   //for(i = 0; i < N; i++){
   //    printf("%d, ", array[i]);
   //}
}
```

Figure 13: C Language Implementation

Machine Code Version

```
.data
array   BYTE 7, 3, 2, 1, 6, 4, 5, 8
last    BYTE 7
temp    BYTE ?

.code
        LOADI  A, 0
Outer:  LOAD   D, [last]
        LOADI  B, 0
        CMP    A, D
        BRGE   End
Inner:  LOAD   D, [last]
        SUB    D, A
        CMP    B, D
        BRGE   Iinc
If:     LOADF  C, [array+B]
        LOADF  D, [array+B+1]
        CMP    D, C
        BRGE   Jinc
Swap:   STOREF [array+B], D
        STOREF [array+B+1], C
Jinc:   ADDI   B, 1
        JUMP   Inner
Iinc:   ADDI   A, 1
        JUMP   Outer
End:    NOOP
```

Figure 14: Assembly Language Implementation

In our i281 CPU, we performed the machine language implementation, by initializing the code memory with the following values:

Assembly v.s. Machine Code

```
.code                                   Code Memory:
        LOADI   A, 0                    0011_00_00_00000000
Outer:  LOAD    D, [last]               1000_11_00_00001000
        LOADI   B, 0                    0011_01_00_00000000
        CMP     A, D                    1101_00_11_00000000
        BRGE    End                     1111_00_11_00001110
Inner:  LOAD    D, [last]               1000_11_00_00001000
        SUB     D, A                    0110_11_00_00000000
        CMP     B, D                    1101_01_11_00000000
        BRGE    Iinc                    1111_00_11_00001000
If:     LOADF   C, [array+B]            1001_10_01_00000000
        LOADF   D, [array+B+1]          1001_11_01_00000001
        CMP     D, C                    1101_11_10_00000000
        BRGE    Jinc                    1111_00_11_00000010
Swap:   STOREF  [array+B], D            1011_11_01_00000000
        STOREF  [array+B+1], C          1011_10_01_00000001
Jinc:   ADDI    B, 1                    0101_01_00_00000001
        JUMP    Inner                   1110_00_00_11110100
Iinc:   ADDI    A, 1                    0101_00_00_00000001
        JUMP    Outer                   1110_00_00_11101110
End:    NOOP                            0000_00_00_00000000
```

Figure 15: Machine Language Implementation

The data memory was then initiated with a list of unordered 8 numbers (from 1 to 8), which finally came out sorted, as shown in the simulation below:
(d0 to d16 represent data memory contents; r0 to r3 represent register file contents; flags_out represents the flags)

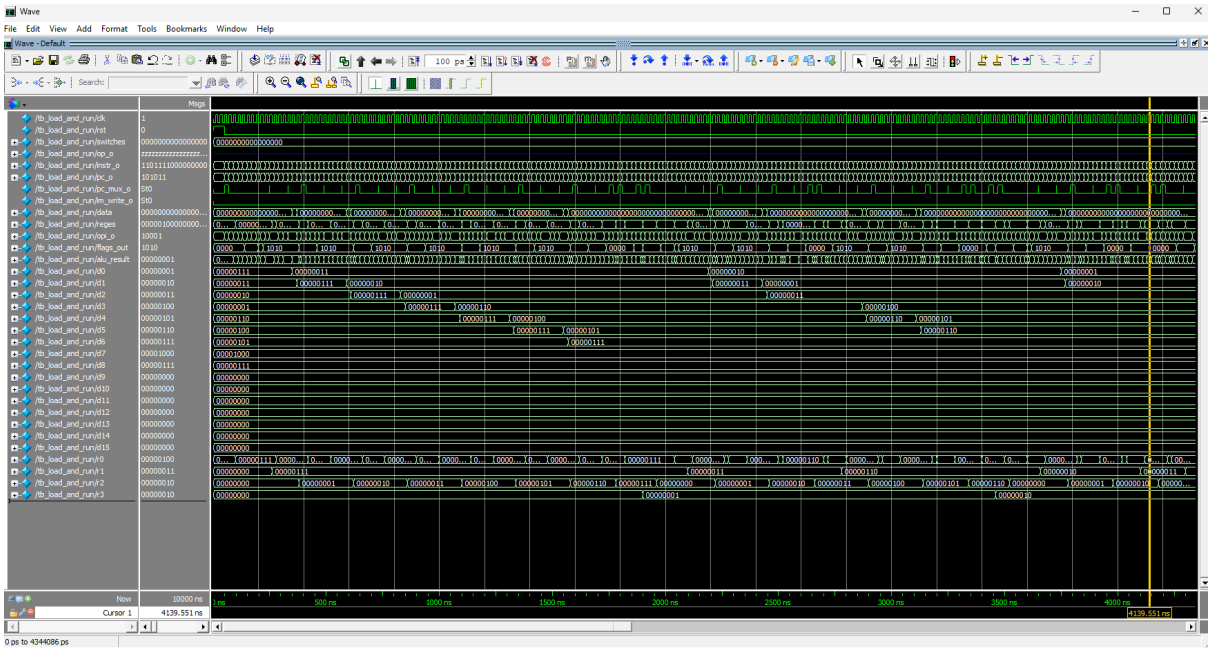


Figure 16: Simulation of Bubble Sort Algorithm